Azure Compute — Hands-on Lab

Virtual Machines (IaaS) and App Service (PaaS)

September 2026

Contents

Introduction & Setup **2**
Hands-on Lab Exercises — Azure Compute . . . 2

Part 1 — Lab Questions **7**
Question 1 — Provision a Linux VM . . . 7
Question 2 — Open a Port with a Network Security Group . . . 9
Question 3 — Resize a VM . . . 10
Question 4 — Attach a Data Disk . . . 11
Question 5 — Deallocate a VM to Stop Paying for Compute . . . 12
Question 6 — Provision a Web Server with a Provisioner Script . . . 13
Question 1 — Deploy a Web App . . . 14
Question 2 — Configure App Settings . . . 15
Question 3 — Scale Out . . . 16
Question 4 — Zero-Downtime Release with a Staging Slot . . . 17

Part 2 — Solutions **18**
Question 1 — Provision a Linux VM . . . 18
Solution — Question 1: Provision a Linux VM . . . 19
Question 2 — Open a Port with a Network Security Group . . . 24
Solution — Question 2: Open a Port with a Network Security Group . . . 24
Question 3 — Resize a VM . . . 28
Solution — Question 3: Resize a VM . . . 28
Question 4 — Attach a Data Disk . . . 31
Solution — Question 4: Attach a Data Disk . . . 31
Question 5 — Deallocate a VM to Stop Paying for Compute . . . 35
Solution — Question 5: Deallocate a VM to Stop Paying for Compute . . . 35
Question 6 — Provision a Web Server with a Provisioner Script . . . 38
Solution — Question 6: Provision a Web Server with a Provisioner Script . . . 38
Question 1 — Deploy a Web App . . . 42
Solution — App Service Q1: Deploy a Web App . . . 42
Question 2 — Configure App Settings . . . 46
Solution — App Service Q2: Configure App Settings . . . 46
Question 3 — Scale Out . . . 49
Solution — App Service Q3: Scale Out . . . 49
Question 4 — Zero-Downtime Release with a Staging Slot . . . 52
Solution — App Service Q4: Zero-Downtime Release with a Staging Slot . . . 52

Part 3 — Practice Exercises **55**
Practice Exercises — Virtual Machines . . . 55
Practice Exercises — App Service . . . 59

Introduction & Setup

Hands-on Lab Exercises — Azure Compute

> **Topic:** Provisioning and managing Azure compute resources — **Virtual Machines** (IaaS) and **App Service** (PaaS).

Welcome. These labs exist to turn you into a **Cloud Administrator** — someone who *manages, maintains, and secures* a company's cloud environment. Not someone who has memorised every command. A real admin doesn't know everything; they know which breadcrumbs to follow. These exercises train exactly that.

Each problem is a short **real-world scenario**. Your job is to work out what it demands and solve it with the tools at your disposal. You get a script to check your work and a full solution walkthrough — freely — because this is a low-stakes place to build competency. Break things here so you don't have to break them in production.

The three skills we're building

1. **Reading documentation** — the first question an admin asks is *"where are the provider's docs for X?"* Learn to navigate them and you can learn any service.
2. **Tooling** — we use the **Azure CLI** (`az`) exclusively. No portal. We like the hard way, because the hard way is the way that sticks.
3. **Troubleshooting** — when things break: composure first. Understand the cause, fix it, and write down what happened.

Prerequisites

- **Azure subscription** — a **paid** (pay-as-you-go) subscription. The labs use small, inexpensive **burstable B-series** VM sizes, chosen because they're among the most widely-available sizes across Azure regions.
- A **lab VM** to work from (set it up below). The Azure CLI runs *there*, not on your laptop.

> **Region tip.** Azure capacity varies by region — a size that's full (or not offered) in one region is usually available in another. If a `create` fails with `SkuNotAvailable` or an allocation error, **just try a different region**: change `LAB_LOCATION` in `lab-config.sh` and re-run. The default is `centralus`; good alternates include `westus2`, `westus3`, `eastus2`, `southcentralus`, `westeurope`, and `northeurope`.

Set up your lab environment

> **This lab is verified on Ubuntu 24.04 LTS Server — the recommended environment for the jumpbox.** Do the exercises from a dedicated Azure lab VM (a "jumpbox") running Ubuntu 24.04 — you'll set it up once and use it all semester. Working from a clean, dedicated VM keeps your own machine tidy and matches how admins actually operate: from a managed box inside the cloud.

*(We use the 24.04 **LTS** for the jumpbox because the Azure CLI's package repo fully supports it. The graded questions still have you provision Ubuntu 26.04 VMs — that's the exercise; the CLI on your jumpbox can create any OS version.)*

1. Provision the lab VM from the Azure Portal. This first VM is your workbench — you'll run every exercise from it. Create it in the **Portal's GUI** (portal.azure.com → **Create a resource** → **Virtual machine**). The Portal is the easiest way to make your very first VM, and it doesn't need the Azure CLI you haven't installed yet.

Create a VM that meets these requirements — the exact size and names are your choice, as long as it satisfies each row:

Setting	Requirement
Image	Ubuntu Server **24.04 LTS** (the Azure CLI installs cleanly on it)
Size	at least **1 vCPU** and **4 GB RAM** — e.g. `Standard_B2als_v2` (2 vCPU / 4 GB)
Authentication type	**Password** (simpler than SSH keys for a first login)
Username	your choice (e.g. `azureuser`)
Password	a strong password *you* set — note it down; you need it to log in
Public inbound ports	allow **SSH (22)** so you can connect
Region	**Central US** works well for this size; if `Standard_B2als_v2` isn't available there, try another location (see the Region tip above)

Put it in its own resource group (e.g. `lab-jumpbox-rg`) — separate from the per-question groups — so it's easy to find all semester and to delete in one action once the course is over.

2. Connect to the lab VM with VS Code. The easiest way to work the labs is **VS Code Remote-SSH**: you get a file explorer, a rendered preview of each `problem.md` / `solution.md`, and a terminal — all running *on the jumpbox*.

New to Remote-SSH? Watch Connect VS Code to a remote server over SSH, or follow Microsoft's step-by-step Remote-SSH tutorial.

One-time setup on your own laptop:

1. Install **VS Code**.
2. In VS Code, open the Extensions view and install **Remote - SSH** (publisher: Microsoft).

Then connect — grab the VM's **Public IP** from its **Overview** page in the Portal, and:

1. Press **F1** (or `Cmd/Ctrl+Shift+P`) → **Remote-SSH: Connect to Host...** → **Add New SSH Host...** → enter `ssh <your-username>@<public-ip>` and save it to your SSH config.
2. Run **Remote-SSH: Connect to Host...** again, pick the host, and enter the **password** you set. A new window opens, connected to the VM.
3. Open a shell on the VM with **Terminal** → **New Terminal** — you'll run the next steps there.

Prefer the command line? Plain `ssh <your-username>@<public-ip>` works too, as does the Portal's **Connect** → **Bastion** (browser-based, no local SSH client).

3. Install the Azure CLI on the lab VM. On Ubuntu, one command does it:

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
az version    # confirm it installed
```

- Azure CLI install (all platforms): https://learn.microsoft.com/cli/azure/install-azure-cli

- Azure CLI install on Linux (Ubuntu/Debian): https://learn.microsoft.com/cli/azure/install-azure-cli-linux

4. Log in. A lab VM has no browser, so use the device-code flow — it prints a URL and a code you enter from any browser:

```
az login --use-device-code
az account show     # confirm the right subscription is active
```

5. Install git and clone this repo onto the lab VM:

```
sudo apt-get update && sudo apt-get install -y git
git clone https://github.com/sailplatform/cloud-admin-lab-exercises-compute.git
cd cloud-admin-lab-exercises-compute
```

6. Open the repo in VS Code and start. In VS Code, **File → Open Folder →** `cloud-admin-lab-exercises-compute`. Now you can browse every question in the explorer. Open a question's `problem.md` and hit the **preview** icon (top-right, or right-click → **Open Preview**) to read it nicely rendered — same for `solution.md`. Then run the scripts in the integrated terminal:

```
cd 1-virtual-machines/Question-1-Provision-Linux-VM
./setup.bash        # then read problem.md and solve it
```

> **Cost warning.** These labs create **real Azure resources that cost real money**. Each question is isolated in its own resource group, and they use small burstable sizes (`Standard_B2als_v2` and friends) that are cheap and widely available. You're billed by the minute, so the golden rule for **question** resources is: provision, solve, validate, then **run `./cleanup.bash` the moment you finish a question.**
>
> **Your lab VM is the exception — keep it for the whole semester; don't delete it.** Just **stop (deallocate)** it when you're done for the day so it stops charging for compute, and **Start** it again next time. Easiest way: in the Portal, open the VM and click **Stop** — that *deallocates* it (compute billing stops; you keep the VM and its disk). Or from any machine with the Azure CLI:
>
> ```
> az vm deallocate --resource-group lab-jumpbox-rg --name <your-vm-name>
> ```
>
> Do **not** "shut down" from inside the OS (`sudo poweroff`) — that leaves the VM **Stopped (allocated)** and **still billing for compute** (the exact trap you'll meet in Question 5). Only **delete** `lab-jumpbox-rg` at the **end of the course**.

How each question is structured

Every graded question lives in its own **self-contained folder** with five files:

File	What it is
`setup.bash`	Prepares/pre-checks the environment for the question.
`problem.md`	The scenario and the exact requirements ("done" criteria).
`validate.bash`	Automated PASS/FAIL checks against your subscription.
`solution.md`	A detailed walkthrough of **how an admin reasons about it** — read this if you get stuck.
`cleanup.bash`	Deletes everything the question created.

`setup.bash`, `validate.bash`, and `cleanup.bash` are executable scripts you run **from inside the question folder** — no central runner, no question numbers to remember. All scripts are open source: read them. Seeing how a check maps to an `az ... --query` is good practice in itself.

The workflow

Everything happens **inside a single question folder** — `cd` into it and go:

```
cd 1-virtual-machines/Question-1-Provision-Linux-VM

./setup.bash       # 1. Prepare/preflight the environment (and, where relevant, provision the
↪  starting VM)

# 2. Read problem.md and solve it yourself with the Azure CLI.
#    Stuck? Open solution.md in the same folder.

./validate.bash    # 3. Check your work - prints PASS/FAIL for each requirement

./cleanup.bash     # 4. Delete the resources so they stop costing money
```

That's the whole model: one folder, three commands (`./setup.bash`, `./validate.bash`, `./cleanup.bash`).

Configuration — make it yours

Shared settings live in `lab-config.sh` at the repo root — most importantly the **region** (`LAB_LOCATION`, default `centralus`). If a region is far from you, lacks capacity for a size, or refuses a deployment, change it there once and every script follows. You can also override for a single run:

```
LAB_LOCATION=westus3 ./validate.bash
```

Hitting capacity or `SkuNotAvailable` errors? That's a region-full situation, not a problem with your account — pick another region and retry. Reliable alternates: `westus2`, `westus3`, `eastus2`, `southcentralus`, `westeurope`, `northeurope`. Resources you already created stay where they are, so also delete any half-created resource group before retrying in a new region.

The practice mindset (why solutions are given freely)

First study the **worked examples** — the graded questions, each with a full `solution.md` walkthrough of how an admin reasons through the task. Then drill the **practice sets** (problems only, no solutions) to build muscle memory and confidence. The point isn't to *know how* to provision a VM — it's to *prove* you can, and to build the habit of finding the answer when you don't already have it.

Available questions

The lab is organized by **compute type**. Each folder below holds its own numbered questions and a `practice/` set — work them from inside each question folder.

`1-virtual-machines/` — Virtual Machines (IaaS)

#	Topic	Skill
1	Provision a Linux VM	Create a resource group + Ubuntu 26.04 VM from scratch
2	Open a Port with an NSG	Diagnose closed connectivity; add an inbound rule
3	Resize a VM	Discover valid sizes; resize in place

#	Topic	Skill
4	Attach a Data Disk	Create + attach a managed data disk
5	Deallocate a VM	Stop compute billing (Stopped vs Deallocated)
6	Provision with a Provisioner Script	cloud-init installs nginx on first boot; curl the public IP

Plus **10 practice exercises** (Windows

- RDP, disk snapshots, Docker on boot, …).

`2-app-services/` — **App Service (PaaS)**

#	Topic	Skill
1	Deploy a Web App	Create an App Service plan + web app on a runtime; curl it live
2	Configure App Settings	Set application settings (env vars) without redeploying
3	Scale Out	Run the app on 3 instances (scale out vs. scale up)
4	Staging Slot + Swap	Zero-downtime release: deploy to a slot, then swap to production

Plus **5 practice exercises** (other runtimes, deploy your own code, connection strings, scale up + Always On, HTTPS-only + log streaming).

Every graded question comes with a full `solution.md` *walkthrough. Practice sets are problems only — your reps for building muscle memory.*

Part 1 — Lab Questions

Question 1 — Provision a Linux VM

Scenario

The development team needs a new Ubuntu Linux server to host an internal tool. As the cloud administrator, **provision a virtual machine that meets their requirements.**

You are on a fresh subscription — nothing has been created for you. Standing up the resource group **and** the VM is your job.

Requirements

Your solution is complete when all of the following are true:

#	Requirement	Value
1	A **resource group** exists	name: `lab-compute-vm01-rg`, in your lab region
2	A **virtual machine** exists in that resource group	name: `dev-tool-vm`
3	The VM runs **Ubuntu 26.04 LTS**	Canonical, offer `ubuntu-26_04-lts`
4	The VM **size**	`Standard_B2als_v2`
5	The **admin username**	`azureuser`
6	Authentication uses an **SSH key**, not a password	password auth disabled
7	The VM is **running**	power state `running`

> **Region:** the resource-group name and region come from `lab-config.sh` at the repo root (default region `centralus`). If you changed `LAB_LOCATION` there, create your resources in that region — the validator reads the same file, so as long as you and it agree, you're fine.

Before you start

Work from **inside this question's folder** (`cd` into it). Then:

```
az login          # authenticate the CLI to your subscription
az account show   # confirm which subscription you're pointed at
./setup.bash           # optional: checks your environment and prints your target
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md` — it walks through **how a cloud administrator reasons about this problem**, not just the final command.

Clean up (avoid surprise charges)

```
./cleanup.bash
```

Question 2 — Open a Port with a Network Security Group

Scenario

The internal tool on `dev-tool-vm` has started serving its web UI on **TCP port 8080**. The team reports they can't reach it from their browsers — the request just times out. As the cloud administrator, **make the tool reachable on port 8080** without opening anything else.

`setup.bash` has already provisioned the VM for you (it exists and is running). Your job is only to open the port.

Requirements

#	Requirement	Value
1	The VM exists and is running	`dev-tool-vm` in `lab-compute-vm02-rg` (setup did this)
2	An **inbound** security rule **allows TCP 8080**	access: Allow, direction: Inbound, port: 8080

> Don't open port ranges or "all ports." A cloud admin opens exactly what's needed — 8080 and nothing more.

Get started

```
./setup.bash      # provisions the VM
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Question 3 — Resize a VM

Scenario

`dev-tool-vm` was provisioned as a `Standard_B2als_v2` (2 vCPU / 4 GB). The tool has grown and the team reports it's CPU-bound during business hours. As the cloud administrator, **give it more capacity by resizing it to `Standard_B4als_v2`** (4 vCPU / 8 GB). Keep the VM running afterwards.

`setup.bash` has provisioned the VM at the smaller size for you.

Requirements

#	Requirement	Value
1	The VM exists	`dev-tool-vm` in `lab-compute-vm03-rg`
2	The VM **size** is now	`Standard_B4als_v2`
3	The VM is **running**	power state `running`

Get started

```
./setup.bash      # provisions the smaller (Standard_B2als_v2) VM
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Question 4 — Attach a Data Disk

Scenario

The internal tool on `dev-tool-vm` is about to start storing a database, and the team wants that data on a **dedicated disk** — separate from the OS disk, so it can grow (and be backed up) independently. As the cloud administrator, **create and attach a new 32 GiB managed data disk** to the VM.

`setup.bash` has provisioned the VM for you.

Requirements

#	Requirement	Value
1	The VM exists	`dev-tool-vm` in `lab-compute-vm04-rg`
2	A **managed data disk** is attached to it	name: `dev-tool-data`
3	Its **size**	32 GiB

> Scope: this is the **cloud-admin** half — provisioning and attaching the disk. Formatting and mounting it *inside* the OS is Linux sysadmin work; the walkthrough points at it, but validation stops at "attached."

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Question 5 — Deallocate a VM to Stop Paying for Compute

Scenario

`dev-tool-vm` is only used during business hours, but it's been running (and billing) around the clock. Finance has asked you to **stop paying for its compute overnight**. As the cloud administrator, put the VM into the state where it **stops incurring compute charges** — without deleting it, so it can be started again tomorrow.

`setup.bash` has provisioned the VM in a running state.

Requirements

#	Requirement	Value
1	The VM still exists (not deleted)	`dev-tool-vm` in `lab-compute-vm05-rg`
2	The VM is in the state that **stops compute billing**	power state `deallocated`

> There's a trap here. "Stopping" a VM from inside the OS (or the wrong CLI move) leaves it **Stopped** but still **allocated** — and you keep paying. The walkthrough explains the difference; the validator checks for the state that actually saves money.

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Question 6 — Provision a Web Server with a Provisioner Script

Scenario

The team wants a web server that comes up **ready to serve** — no manual SSH, no hand-installing packages. As the cloud administrator, **provision a VM that configures itself on first boot**: a provisioner script should install and start **nginx**, and the VM should be reachable on **port 80** so that curling its public IP returns the nginx welcome page.

This is how real deployments work: the VM is described *as code* and stands itself up. `setup.bash` only checks your environment — building the VM (with its provisioner script) is the whole exercise.

Requirements

#	Requirement	Value
1	A VM exists	`dev-tool-vm` in `lab-compute-vm06-rg`, Ubuntu 26.04, `Standard_B2als_v2`
2	**nginx** is installed and running — placed there by a **provisioner script**, not by manual SSH	via cloud-init `--custom-data`
3	Inbound **TCP 80** is open	access: Allow, direction: Inbound, port: 80
4	Curling the VM's public IP returns nginx	`curl http://<public-ip>` → HTTP 200, `Server: nginx`

> A provisioner script runs on **first boot**, so give it a minute or two after the VM is created before you validate — installing nginx isn't instant.

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Question 1 — Deploy a Web App

Scenario

The team wants to stop managing servers for a small internal website. Instead of a VM they'll patch and babysit, they want a **platform-managed web app** — they push code, Azure runs it. As the cloud administrator, **stand up an Azure App Service web app** and prove it's live in a browser.

App Service is a **Platform-as-a-Service (PaaS)**: you don't see the VM, the OS, or the web server — Azure runs all of that. You provide the app; Azure provides the platform.

Requirements

#	Requirement	Value
1	A resource group exists	`lab-compute-app01-rg`, in your lab region
2	An **App Service plan** exists in it	Linux, a **paid** tier (e.g. `B1` Basic) — not Free (`F1`)
3	A **web app** runs on that plan	a **Node** runtime (e.g. `NODE:22-lts`)
4	The app is **reachable**	`curl https://<app-name>.azurewebsites.net` returns **HTTP 200**

> **Web app names are globally unique** — the name becomes `https://<name>.azurewebsites.net`, a public DNS host shared by all of Azure. Pick something unlikely to be taken, e.g. `dev-tool-app-<your-initials><digits>`. The validator finds *the* web app in your resource group, so the exact name is your choice.

Before you start

Work from **inside this question's folder**. Then:

```
az login
az account show
./setup.bash      # optional: preflight + shows your target
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Question 2 — Configure App Settings

Scenario

The web app needs configuration: it reads its behaviour from environment variables (which database to use, which mode to run in, feature toggles). In App Service these are called **application settings** — key/value pairs Azure injects into the app's environment. As the cloud administrator, **set the app's configuration** without redeploying any code.

`setup.bash` has already provisioned a plan and a running web app for you.

Requirements

The web app in `lab-compute-app02-rg` must have these two application settings:

Setting name	Value
`ENVIRONMENT`	`production`
`FEATURE_FLAG`	`beta`

> App settings are per-app configuration, surfaced to your code as environment variables. Setting them restarts the app automatically — that's expected.

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Question 3 — Scale Out

Scenario

A marketing push is coming and the single instance behind the web app won't keep up. Rather than move to a bigger machine, the team wants **more copies of the app running behind Azure's load balancer** so traffic is spread across them. As the cloud administrator, **scale the app out to 3 instances.**

`setup.bash` has provisioned the plan and web app (currently 1 instance).

Requirements

#	Requirement	Value
1	A web app exists	in `lab-compute-app03-rg`
2	The App Service plan runs **3 instances**	worker/instance count = `3`

> "Scale **out**" (more instances) is different from "scale **up**" (a bigger tier). This question is about scaling *out*. The Basic tier supports up to 3 instances — enough for this exercise.

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Question 4 — Zero-Downtime Release with a Staging Slot

Scenario

Production is live and you need to ship a new version **without downtime** and **without risk**. App Service's answer is **deployment slots**: a separate, fully-running copy of the app (e.g. `staging`) that you deploy and test on its own URL, then **swap** into production — Azure warms up the new version first and flips traffic over instantly.

`setup.bash` has provisioned the plan (Standard tier — slots require it) and a web app whose current release is marked `RELEASE=v1`.

Your job: roll out **v2** through a staging slot and swap it live.

Requirements

#	Requirement	Value
1	A deployment slot named **`staging`** exists on the web app	slot `staging`
2	You mark the new release in staging and **swap** it into production	prod app setting `RELEASE` = `v2` after the swap
3	Production is healthy after the swap	`curl https://<app>.azurewebsites.net` → HTTP 200

> We use the app setting `RELEASE` as a stand-in for "a new version of the code": set `RELEASE=v2` on the **staging** slot, confirm it there, then swap — after which **production** carries `v2`. (App settings move with the slot on swap unless you pin them as slot-specific.)

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Part 2 — Solutions

Question 1 — Provision a Linux VM

Scenario

The development team needs a new Ubuntu Linux server to host an internal tool. As the cloud administrator, **provision a virtual machine that meets their requirements.**

You are on a fresh subscription — nothing has been created for you. Standing up the resource group **and** the VM is your job.

Requirements

Your solution is complete when all of the following are true:

#	Requirement	Value
1	A **resource group** exists	name: `lab-compute-vm01-rg`, in your lab region
2	A **virtual machine** exists in that resource group	name: `dev-tool-vm`
3	The VM runs **Ubuntu 26.04 LTS**	Canonical, offer `ubuntu-26_04-lts`
4	The VM **size**	`Standard_B2als_v2`
5	The **admin username**	`azureuser`
6	Authentication uses an **SSH key**, not a password	password auth disabled
7	The VM is **running**	power state `running`

> **Region:** the resource-group name and region come from `lab-config.sh` at the repo root (default region `centralus`). If you changed `LAB_LOCATION` there, create your resources in that region — the validator reads the same file, so as long as you and it agree, you're fine.

Before you start

Work from **inside this question's folder** (`cd` into it). Then:

```
az login          # authenticate the CLI to your subscription
az account show   # confirm which subscription you're pointed at
./setup.bash           # optional: checks your environment and prints your target
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md` — it walks through **how a cloud administrator reasons about this problem**, not just the final command.

Clean up (avoid surprise charges)

```
./cleanup.bash
```

Solution — Question 1: Provision a Linux VM

Before you skip to the command at the bottom: the command is the *least* important thing here. Anyone can copy a command. What makes you a cloud administrator is the thinking that happens before you type it. That's what this walkthrough is really about.

Step 1 — Read the problem like an admin

A seasoned admin doesn't read a ticket looking for a command. They read it looking for **constraints** — the non-negotiable facts the finished resource must satisfy. Underline the nouns and adjectives:

> Provision a **virtual machine**, running **Ubuntu 26.04 LTS**, size **Standard_B2als_v2**, admin user **azureuser**, using an **SSH key** (not a password), and leave it **running**, inside a **resource group** named `lab-compute-vm01-rg`.

Every highlighted word is going to become one flag on one command. Turn the prose into a checklist:

Keyword in the problem	What it becomes
resource group `lab-compute-vm01-rg`	a container you create first
virtual machine `dev-tool-vm`	the thing you're provisioning
Ubuntu 26.04 LTS	which OS image to pick
Standard_B2als_v2	the VM size (CPU/RAM tier)
azureuser	the admin username
SSH key, not a password	the authentication method
running	the expected end state

You now know your destination *exactly*. Half of troubleshooting is just knowing precisely what "done" looks like — you just wrote it down.

Step 2 — Decide the shape before touching the keyboard

Two facts aren't spelled out as flags but you still have to settle them:

- **Where?** Every Azure resource lives in a region. This lab keeps the region in `lab-config.sh` (`LAB_LOCATION`, default `centralus`) so every script agrees. Real admins pick a region for latency, cost, and data-residency reasons.
- **What has to exist first?** In Azure a VM must live inside a **resource group**. Nothing here creates it for you, so that's step one. A resource group is just a labelled box — its superpower is that deleting the box deletes everything in it (which is exactly how `cleanup.bash` works).

Mental model: **create the box → put the VM in the box.**

A word on the size. The problem hands you `Standard_B2als_v2`, but understand *why*. It's a **burstable B-series** size (2 vCPU / 4 GB): inexpensive, and among the **most widely-available sizes across Azure regions**. Bigger or specialized families are more often **capacity-constrained** — Azure replies `SkuNotAvailable` or an allocation error when a region is temporarily full of that size. Burstable sizes bank CPU credits while idle and spend them under load, a great fit for labs and light workloads. Rule of thumb: if a `create` fails, it's usually the **region's capacity**, not your account — try another region by changing `LAB_LOCATION` in `lab-config.sh` (`westus2`, `westus3`, `eastus2`, and `westeurope` are reliable alternates).

Step 3 — Find the reference (the real skill)

You are *not* expected to remember VM flags. You're expected to know **where the answer lives** and how to get there fast. The path is always the same:

Azure CLI reference → Virtual Machines → Commands → `az vm` → `az vm create`

Direct link: https://learn.microsoft.com/en-us/cli/azure/vm?view=azure-cli-latest

Notice the shape of the `az` command tree — it reads like a sentence:

```
az   vm   create
|    |     `- the action (verb)
|    `- the resource (noun)
`- the tool
```

Almost every Azure task you'll ever do is `az <resource> <action>`. Learn to *guess* the noun and verb, then confirm in the docs. Need to create a group? `az group create`. Start a VM? `az vm start`. Resize one? `az vm resize`. The pattern is the documentation's table of contents.

Step 4 — Tame the parameter overload with `--help`

Open the `az vm create` page (or run `az vm create --help`) and your first reaction is fair: **there are dozens of parameters.** Nobody uses all of them. Here's how an admin cuts a wall of flags down to size:

```
az vm create --help
```

Read it with two filters:

1. **What's required?** The help marks required arguments. For `az vm create` the essentials are just `--resource-group` and `--name`. Everything else has a sensible default.
2. **Which optional flags map to MY constraints?** Go back to your Step-1 checklist and, for each item, scan the help text for the matching flag:

My constraint	Flag
Ubuntu 26.04	`--image <URN>` (see Step 4b — no shortcut alias exists for 26.04)
Standard_B2als_v2	`--size Standard_B2als_v2`
azureuser	`--admin-username azureuser`
SSH key, no password	`--generate-ssh-keys` (creates a keypair AND disables password login)

Ignore the other 40 flags. They exist for scenarios you don't have. This filter — *required first, then match the rest to your checklist* — is how you stay calm in front of any over-parameterised command.

> **Tip:** `--generate-ssh-keys` reuses an existing key in `~/.ssh` if you have one, or creates a fresh keypair if you don't. Because it uses key-based auth, Azure sets `disablePasswordAuthentication = true` for you — requirement #6 handled without a separate flag.

Step 4b — When there's no image alias, build the URN yourself

For common images the CLI offers short **aliases**. List them:

```
az vm image list --output table   # look at the "UrnAlias" column
```

You'll see `Ubuntu2204`, `Ubuntu2404`… but **no `Ubuntu2604`**. Aliases lag behind new releases. This is the exact moment novices get stuck — and the exact moment an admin shrugs and finds the image the long way. Every Azure image has a four-part identifier called a **URN**:

`Publisher : Offer : Sku : Version`

You discover each part with three `az vm image` sub-commands, narrowing as you go:

```
# 1) Publisher - Ubuntu images come from Canonical
az vm image list-publishers --location centralus -o table | grep -i canonical

# 2) Offer - find the 26.04 LTS offer
az vm image list-offers --publisher Canonical --location centralus -o table | grep -i 26
#   -> ubuntu-26_04-lts

# 3) Sku - pick the plain server image (this is Gen2; 'server-gen1' is the old Gen1)
az vm image list-skus --publisher Canonical --offer ubuntu-26_04-lts --location centralus -o table
#   -> server, server-gen1, minimal, ...
```

Assemble the parts, using `latest` for the version so you always get the newest patch:

`Canonical:ubuntu-26_04-lts:server:latest`

That string goes straight into `--image`. You just did what the alias would have done — and now you can find *any* image, aliased or not. (The `server` SKU is Gen2, which is what current sizes like `Standard_B2als_v2` expect.)

Step 5 — Build the command incrementally

Assemble it flag by flag, each one traceable to a line in your checklist. Create the box first:

```
# 1) Create the resource group (the box). Region comes from lab-config.sh.
az group create \
  --name lab-compute-vm01-rg \
  --location centralus
```

```
# 2) Create the VM inside it, one flag per constraint.
az vm create \
  --resource-group lab-compute-vm01-rg \                # which box
  --name           dev-tool-vm \                        # the VM's name
  --image          Canonical:ubuntu-26_04-lts:server:latest \  # Ubuntu 26.04 LTS (Gen2)
  --size           Standard_B2als_v2 \                    # the CPU/RAM tier
  --admin-username azureuser \                          # the login name
  --generate-ssh-keys                                   # SSH-key auth, password disabled
```

`az vm create` provisions the VM **and** starts it, so requirement #7 (running) is satisfied the moment the command returns. It prints a small JSON summary including the `publicIpAddress` — hold onto that if you want to SSH in.

If you changed `LAB_LOCATION` in `lab-config.sh`, use *that* region in the `az group create --location` above. The two must match.

Step 6 — Verify with your own eyes

Don't trust "it didn't error" — confirm the facts. `az vm show` prints the VM's definition; `--query` pulls out just the field you care about (that's JMESPath, the query language the CLI uses):

```
# Is it the size and image I asked for?
az vm show -g lab-compute-vm01-rg -n dev-tool-vm \
  --query "{size:hardwareProfile.vmSize, offer:storageProfile.imageReference.offer}" -o table

# Is it actually running?
az vm get-instance-view -g lab-compute-vm01-rg -n dev-tool-vm \
  --query "instanceView.statuses[?starts_with(code, 'PowerState/')].displayStatus" -o tsv

# What's its public IP (for SSH)?
az vm list-ip-addresses -g lab-compute-vm01-rg -n dev-tool-vm -o table
```

Then let the graded check confirm everything at once:

```
./validate.bash
```

Step 7 — When it goes wrong, stay composed

Errors are information, not emergencies. Read the message; it usually names the fix. The common ones here:

Symptom	What it means	Move
`Please run 'az login'`	The CLI isn't authenticated	`az login`, then retry
`ResourceGroupNotFound`	You skipped step 1, or misspelled the group	Create the group first; check the name matches exactly
`The platform image ... is not available` / bad URN	A typo in the four-part URN, or wrong region	Re-run the Step-4b discovery commands and copy the parts exactly
`SkuNotAvailable` / allocation error for the size	That size is temporarily out of capacity in your region (or not offered there)	Change `LAB_LOCATION` in `lab-config.sh` to another region (`westus2`, `westus3`, `eastus2`, `westeurope`) and retry; or pick another **B-series v2** size (`Standard_B2ats_v2`, `Standard_B2as_v2`)
VM name/DNS already taken	The name collides with something existing	Delete the old resource or pick a clean region

A real admin's habit: when something breaks, write down *what* you saw and *how* you fixed it. That note is worth more next time than any memorised command.

Cheat sheet

Goal	Command
Log in	`az login`
Create a resource group	`az group create -n <rg> -l <region>`
List image aliases	`az vm image list -o table`
Find an image offer/sku	`az vm image list-offers --publisher Canonical -l <region>` / `... list-skus --offer <offer>`
Create the VM	`az vm create -g <rg> -n <vm> --image Canonical:ubuntu-26_04-lts:server:latest --size Standard_B2als_v2 --admin-username azureuser --generate-ssh-keys`
Inspect a VM	`az vm show -g <rg> -n <vm> -o jsonc`
Check power state	`az vm get-instance-view -g <rg> -n <vm> --query "instanceView.statuses[?starts_with(code,'PowerState/')].displayStatus" -o tsv`
Get the public IP	`az vm list-ip-addresses -g <rg> -n <vm> -o table`
See a command's options	`az vm create --help`
Delete everything	`az group delete -n <rg> --yes --no-wait`

The transferable takeaway: read the ticket into a checklist → find the `az <noun> <verb>` in the reference → use `--help` to match flags to your checklist (and `az vm image list-*` to build a URN when no alias exists) → build incrementally → verify each fact. That loop solves the next VM question, and the App Service one after it, and services you've never seen.

Question 2 — Open a Port with a Network Security Group

Scenario

The internal tool on `dev-tool-vm` has started serving its web UI on **TCP port 8080**. The team reports they can't reach it from their browsers — the request just times out. As the cloud administrator, **make the tool reachable on port 8080** without opening anything else.

`setup.bash` has already provisioned the VM for you (it exists and is running). Your job is only to open the port.

Requirements

#	Requirement	Value
1	The VM exists and is running	`dev-tool-vm` in `lab-compute-vm02-rg` (setup did this)
2	An **inbound** security rule **allows TCP 8080**	access: Allow, direction: Inbound, port: 8080

> Don't open port ranges or "all ports." A cloud admin opens exactly what's needed — 8080 and nothing more.

Get started

```
./setup.bash      # provisions the VM
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Solution — Question 2: Open a Port with a Network Security Group

The command here is one line. The point of the exercise is understanding **why the port was closed in the first place** and **which knob controls it** — because that understanding is what lets you debug connectivity for the rest of your career.

Step 1 — Diagnose before you change anything

The team says the request "just times out." A timeout (as opposed to "connection refused") is a strong signal: something is silently **dropping** the packets before they reach the app. In Azure, that "something" is almost always a **Network Security Group (NSG)** — a stateful firewall attached to the VM's network interface (or its subnet).

When `az vm create` builds a VM, it creates an NSG that allows **SSH (22)** and denies everything else inbound by default. Your web app on 8080 was never allowed in. Nothing is broken — the default is simply "closed unless you say otherwise." That default-deny posture is a security feature, not a bug.

Mental model: **the NSG is a bouncer with an allow-list. Port 8080 isn't on the list yet.**

Step 2 — Turn the requirement into constraints

Keyword in the problem	Constraint
reachable on 8080	inbound rule, destination port 8080
TCP	protocol TCP
"not opening anything else"	one specific port — no ranges, no "*"
inbound (browser → VM)	direction = Inbound, access = Allow

Step 3 — Find the right command

Guess the noun and verb. You're touching a VM's networking, so start at `az vm` and look for something about ports:

```
az vm --help | grep -i port
```

You'll find **`az vm open-port`** — a convenience command that finds the VM's NSG and adds the allow rule for you. That's the fast path. Reference: https://learn.microsoft.com/cli/azure/vm#az-vm-open-port

There is also the lower-level, more explicit path through the network commands (`az network nsg rule create`), which every serious admin should recognize: https://learn.microsoft.com/cli/azure/network/nsg/rule#az-network-nsg-rule-create

---

Step 4 — Do it (the quick way)

```
az vm open-port \
  --resource-group lab-compute-vm02-rg \
  --name           dev-tool-vm \
  --port           8080 \
  --priority       900        # lower number = evaluated first; must be unique in the NSG
```

`--priority` matters because NSG rules are evaluated **in order** (100 → 4096) and the first match wins. Pick a value that doesn't collide with the existing SSH rule.

The explicit way (worth knowing)

This is what `open-port` does under the hood — spelled out. First find the NSG, then add a precise rule:

```
# The NSG az created is named after the VM:
az network nsg list -g lab-compute-vm02-rg --query "[].name" -o tsv
# -> dev-tool-vmNSG

az network nsg rule create \
  --resource-group        lab-compute-vm02-rg \
  --nsg-name              dev-tool-vmNSG \
  --name                  allow-8080 \
```

```
  --priority              900 \
  --direction             Inbound \
  --access                Allow \
  --protocol              Tcp \
  --destination-port-ranges 8080
```

Notice how each flag maps straight to a constraint from Step 2. When a convenience command doesn't fit your exact need, dropping down to the explicit command is the move — and now the "wall of flags" reads as your own checklist.

Step 5 — Verify

```
# See the rule you just added
az network nsg rule list \
  -g lab-compute-vm02-rg --nsg-name dev-tool-vmNSG \
  --query "[?destinationPortRange=='8080' || contains(destinationPortRanges, '8080')].{name:name,
   ↪  access:access, dir:direction, port:destinationPortRange}" \
  -o table

./validate.bash
```

To truly confirm end-to-end you'd also need the app actually listening on 8080 inside the VM — but the cloud-admin responsibility here is the network path, and that's what we validate.

Step 6 — Troubleshooting with composure

Symptom	Likely cause	Move
`Priority NNN already exists`	Two rules share a priority	Pick a different, unique `--priority`
Rule added but still times out	App isn't listening on 8080 inside the VM, or a **subnet-level** NSG also blocks it	`curl localhost:8080` on the VM; check for an NSG on the subnet too
Can SSH but not reach 8080	Only 22 was ever allowed	That's exactly this task — add the 8080 rule

Security reflex: every time you open a port, ask *"from where?"* Production admins scope `--source-address-prefixes` to known IP ranges instead of the whole internet. This lab keeps it simple, but the habit is the lesson.

Cheat sheet

Goal	Command
Open a port (quick)	`az vm open-port -g <rg> -n <vm> --port 8080 --priority 900`
List a VM's NSGs	`az network nsg list -g <rg> --query "[].name" -o tsv`

Goal	Command
Add an explicit rule	`az network nsg rule create -g <rg> --nsg-name <nsg> -n allow-8080 --priority 900 --direction Inbound --access Allow --protocol Tcp --destination-port-ranges 8080`
List rules	`az network nsg rule list -g <rg> --nsg-name <nsg> -o table`
Delete a rule	`az network nsg rule delete -g <rg> --nsg-name <nsg> -n allow-8080`

Takeaway: connectivity problems in the cloud are usually *permission* problems, not *plumbing* problems. Learn where the allow-lists live (NSGs), and "it times out" stops being scary.

Question 3 — Resize a VM

Scenario

`dev-tool-vm` was provisioned as a `Standard_B2als_v2` (2 vCPU / 4 GB). The tool has grown and the team reports it's CPU-bound during business hours. As the cloud administrator, **give it more capacity by resizing it to `Standard_B4als_v2`** (4 vCPU / 8 GB). Keep the VM running afterwards.

`setup.bash` has provisioned the VM at the smaller size for you.

Requirements

#	Requirement	Value
1	The VM exists	`dev-tool-vm` in `lab-compute-vm03-rg`
2	The VM **size** is now	`Standard_B4als_v2`
3	The VM is **running**	power state `running`

Get started

```
./setup.bash      # provisions the smaller (Standard_B2als_v2) VM
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Solution — Question 3: Resize a VM

Resizing is one command. The skill being tested is knowing **what sizes are actually available to you** before you commit to one — because the size you want isn't always offered in your region, and picking blindly is how you hit a wall mid-change.

Step 1 — Read the requirement into constraints

Keyword in the problem	Constraint
CPU-bound, needs more capacity	move to a bigger size
Standard_B4als_v2	the exact target size (4 vCPU / 8 GB)
keep it running	the VM should be `running` when you're done

The problem hands you the target, but in real life *you* choose it. So the first real skill is: **how do I find a valid larger size?**

Step 2 — Discover the available sizes (don't guess)

Two commands answer "what can I resize this VM to?"

```
# What sizes can THIS VM be resized to, in place?
az vm list-vm-resize-options \
  --resource-group lab-compute-vm03-rg --name dev-tool-vm \
  --query "[].name" -o table
```

```
# Or, more generally: what sizes exist in the region at all?
az vm list-sizes --location centralus \
  --query "[?starts_with(name, 'Standard_B')].{name:name, vCPUs:numberOfCores, memMB:memoryInMb}" \
  -o table
```

`Standard_B4als_v2` is the same family (burstable **B-series v2**) as the current `Standard_B2als_v2`, just a bigger tier — the safest kind of resize, because same-family moves rarely require special hardware and usually complete without surprises. Reference: https://learn.microsoft.com/cli/azure/vm#az-vm-resize

---

Step 3 — Understand what resizing actually does

A seasoned admin knows the catch **before** running the command:

- If the target size is available on the **current host**, the resize applies with a quick reboot.
- If it isn't, Azure must **deallocate** the VM, move it to a host that supports the size, and start it again — a short outage. Same-family, adjacent-tier moves (like ours) almost always stay on the same host.

So resizing can briefly interrupt the service. You'd schedule it in a maintenance window in production. Here, just know that a reboot may happen.

Step 4 — Do it

```
az vm resize \
  --resource-group lab-compute-vm03-rg \
  --name           dev-tool-vm \
  --size           Standard_B4als_v2
```

`az vm resize` leaves the VM in its prior power state — since setup left it running, it comes back running, satisfying requirement #3. (If a resize ever leaves it stopped, `az vm start -g <rg> -n <vm>` brings it back.)

Step 5 — Verify

```
az vm show -g lab-compute-vm03-rg -n dev-tool-vm \
  --query hardwareProfile.vmSize -o tsv          # -> Standard_B4als_v2

az vm get-instance-view -g lab-compute-vm03-rg -n dev-tool-vm \
  --query "instanceView.statuses[?starts_with(code,'PowerState/')].displayStatus" -o tsv

./validate.bash
```

Step 6 — Troubleshooting with composure

Symptom	Likely cause	Move
`SkuNotAvailable` / size not in the resize list	The target size isn't offered on the current host or region	Use `az vm list-vm-resize-options`; if empty, `az vm deallocate` then resize then `az vm start`
Resize succeeds but VM is stopped	Azure had to relocate it	`az vm start -g <rg> -n <vm>`
Quota error	Your subscription's vCPU quota for the family is exhausted	Request a quota increase, or pick a smaller/other family

The mindset: capacity changes are cheap to *do* and easy to *reverse* — you can resize back down just as fast. What separates an admin is checking availability first and knowing a reboot might happen, so nobody is surprised.

Cheat sheet

Goal	Command
List valid in-place resize targets	`az vm list-vm-resize-options -g <rg> -n <vm> --query "[].name" -o table`
List all sizes in a region	`az vm list-sizes -l <region> -o table`
Resize	`az vm resize -g <rg> -n <vm> --size Standard_B4als_v2`
Confirm size	`az vm show -g <rg> -n <vm> --query hardwareProfile.vmSize -o tsv`
Start (if it came back stopped)	`az vm start -g <rg> -n <vm>`

Takeaway: never hard-code a size you haven't confirmed is available. The discovery commands (`list-vm-resize-options`, `list-sizes`) are the difference between a smooth change and a stuck one.

Question 4 — Attach a Data Disk

Scenario

The internal tool on `dev-tool-vm` is about to start storing a database, and the team wants that data on a **dedicated disk** — separate from the OS disk, so it can grow (and be backed up) independently. As the cloud administrator, **create and attach a new 32 GiB managed data disk** to the VM.

`setup.bash` has provisioned the VM for you.

Requirements

#	Requirement	Value
1	The VM exists	`dev-tool-vm` in `lab-compute-vm04-rg`
2	A **managed data disk** is attached to it	name: `dev-tool-data`
3	Its **size**	32 GiB

> Scope: this is the **cloud-admin** half — provisioning and attaching the disk. Formatting and mounting it *inside* the OS is Linux sysadmin work; the walkthrough points at it, but validation stops at "attached."

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Solution — Question 4: Attach a Data Disk

Attaching a disk is one command with a small twist: you can create the disk and attach it in a single step, or create it first and attach it second. Knowing *why* you'd separate OS data from application data is the real lesson.

Step 1 — Read the requirement into constraints

Keyword in the problem	Constraint
dedicated disk, separate from OS	a **data** disk (not the OS disk)
managed	a **managed** disk (Azure handles the storage account)

Keyword in the problem	Constraint
32 GiB	`--size-gb 32`
named `dev-tool-data`	`--name dev-tool-data`

Why a separate data disk? The OS disk should stay small and disposable — you might rebuild the VM. Data that must survive a rebuild (a database, uploads, logs) belongs on its own disk you can detach and re-attach, snapshot, or resize independently. This separation is a core operational habit.

Step 2 — Find the command

You're acting on a VM's disks, so the noun/verb guess is `az vm disk`:

```
az vm disk --help
```

You'll find **`az vm disk attach`**. Read its `--help` and two flags stand out:

- `--new` — create a brand-new disk instead of attaching an existing one.
- `--size-gb` — the size, used together with `--new`.

Reference: https://learn.microsoft.com/cli/azure/vm/disk#az-vm-disk-attach

---

Step 3 — Do it (create + attach in one step)

```
az vm disk attach \
  --resource-group lab-compute-vm04-rg \
  --vm-name        dev-tool-vm \
  --name           dev-tool-data \
  --new \
  --size-gb        32
```

`--new` tells Azure to provision a fresh managed disk named `dev-tool-data` and attach it to the VM in one call. (Without `--new`, `--name` must refer to an existing disk — that's how you'd re-attach a disk you detached earlier.)

The two-step way (worth recognizing)

Sometimes you create the disk first — e.g. to set its SKU or zone — then attach:

```
az disk create -g lab-compute-vm04-rg -n dev-tool-data --size-gb 32 --sku Premium_LRS
az vm disk attach -g lab-compute-vm04-rg --vm-name dev-tool-vm --name dev-tool-data
```

Same result; more control. Choosing the one-liner vs. the two-step is exactly the kind of judgment call the docs let you make once you know both exist.

Step 4 — Verify (the cloud-admin part)

```
# The disk appears in the VM's data-disk list (name + LUN):
az vm show -g lab-compute-vm04-rg -n dev-tool-vm \
  --query "storageProfile.dataDisks[].{name:name, lun:lun}" -o table

# Its SIZE lives on the disk resource, not the VM's entry (that inline field is
```

```
# usually blank for managed disks). The property is diskSizeGb OR diskSizeGB
# depending on your CLI version - this handles both:
az disk show -g lab-compute-vm04-rg -n dev-tool-data --query "diskSizeGb || diskSizeGB" -o tsv

./validate.bash
```

`lun` (Logical Unit Number) is the slot the disk occupies on the VM — you'll use it to identify the disk inside the OS. Note the split: the **VM** records *that* a disk is attached and *where* (LUN); the disk's *size* and other properties live on the **disk resource** you query with `az disk show`.

Step 5 — The other half: make the OS use it (further reading)

Attaching gives the VM a **raw block device**. Linux still has to partition, format, and mount it before anything can store data there. This is sysadmin work, not cloud-admin work, so it's outside what we validate — but here's the shape so you're not surprised.

First, find the device — don't assume `/dev/sdc`. Azure's Linux device names vary by VM size: the data disk might be `/dev/sdc`, another letter, or an NVMe device like `/dev/nvme0n2`. Guessing a name that doesn't exist gives `Error: Could not stat device /dev/sdc - No such file or directory`. So list them:

```
# (SSH into the VM first)
lsblk -o NAME,SIZE,TYPE,MOUNTPOINTS    # find the ~32G disk of TYPE 'disk' with NO mountpoint
```

The reliable way is Azure's stable LUN symlink — it points to the right disk no matter what the kernel calls it. A data disk attached at LUN 0 is always `/dev/disk/azure/scsi1/lun0`:

```
ls -l /dev/disk/azure/scsi1/           # data disks show as lun0, lun1, ...

sudo parted /dev/disk/azure/scsi1/lun0 --script mklabel gpt mkpart primary ext4 0% 100%
sudo mkfs.ext4 /dev/disk/azure/scsi1/lun0-part1
sudo mkdir -p /data
sudo mount /dev/disk/azure/scsi1/lun0-part1 /data
# make it permanent by adding it to /etc/fstab (use the disk's UUID from `blkid`)
```

> If `/dev/disk/azure/scsi1/` doesn't exist on your image, fall back to the device name you found with `lsblk` — e.g. `sudo parted /dev/sdX ...` then `sudo mkfs.ext4 /dev/sdX1`.

Cloud provisions the hardware; the OS makes it usable. Knowing where that line sits — and that device names aren't fixed — is part of being a well-rounded admin.

Step 6 — Troubleshooting with composure

Symptom	Likely cause	Move
`already exists` on `--new`	A disk named `dev-tool-data` already exists	Drop `--new` to attach the existing one, or pick a new name
`Could not stat device /dev/sdc - No such file or directory`	`/dev/sdc` was just an example — your disk has a different name (Azure device naming varies; may be NVMe)	Find the real device with `lsblk`, or use the stable `/dev/disk/azure/scsi1/lun0` path (Step 5)
Disk attached but not visible in the OS	You haven't partitioned/mounted it yet	Do Step 5 inside the VM

Symptom	Likely cause	Move
Wrong size shows up	`--size-gb` typo, or you attached an existing disk of another size	Detach (`az vm disk detach`), fix, re-attach

---

Cheat sheet

Goal	Command
Create + attach a new data disk	`az vm disk attach -g <rg> --vm-name <vm> --name <disk> --new --size-gb 32`
Attach an existing disk	`az vm disk attach -g <rg> --vm-name <vm> --name <disk>`
List a VM's data disks	`az vm show -g <rg> -n <vm> --query "storageProfile.dataDisks[].{name:name,lun:lun}" -o table`
Check a disk's size	`az disk show -g <rg> -n <disk> --query "diskSizeGb || diskSizeGB" -o tsv`
Detach a disk	`az vm disk detach -g <rg> --vm-name <vm> --name <disk>`
Create a standalone disk	`az disk create -g <rg> -n <disk> --size-gb 32 --sku Premium_LRS`

Takeaway: separate the disposable (OS) from the durable (data). Attaching the disk is the cloud-admin move; formatting it is the OS move — and a good admin knows exactly where that boundary is.

Question 5 — Deallocate a VM to Stop Paying for Compute

Scenario

`dev-tool-vm` is only used during business hours, but it's been running (and billing) around the clock. Finance has asked you to **stop paying for its compute overnight**. As the cloud administrator, put the VM into the state where it **stops incurring compute charges** — without deleting it, so it can be started again tomorrow.

`setup.bash` has provisioned the VM in a running state.

Requirements

#	Requirement	Value
1	The VM still exists (not deleted)	`dev-tool-vm` in `lab-compute-vm05-rg`
2	The VM is in the state that **stops compute billing**	power state `deallocated`

> There's a trap here. "Stopping" a VM from inside the OS (or the wrong CLI move) leaves it **Stopped** but still **allocated** — and you keep paying. The walkthrough explains the difference; the validator checks for the state that actually saves money.

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Solution — Question 5: Deallocate a VM to Stop Paying for Compute

This question looks trivial — "turn it off" — and that's exactly the trap. The lesson is one of the most valuable (and most commonly botched) facts in cloud cost management: **Stopped is not the same as Deallocated.**

Step 1 — The concept that makes or breaks your bill

A VM has three "off-ish" states, and only one stops the compute meter:

State	What it means	Are you billed for compute?
Running	Powered on	**Yes**
Stopped (a.k.a. "Stopped (allocated)")	Guest OS is halted, but Azure still **reserves the CPU/RAM** for you	**Yes — still billing!**
Deallocated ("Stopped (deallocated)")	The VM is released from its host; no compute reserved	**No** (you still pay for disks)

Here's how people get burned: running `shutdown` / `poweroff` **inside** the VM, or `az vm stop`, leaves it **Stopped (allocated)** — the OS is off but Azure is still holding (and charging for) the hardware. To actually stop paying, you must **deallocate**, which hands the hardware back to Azure.

> You still pay for the **disks** while deallocated — storage is cheap and persistent. It's the **compute** (the expensive part) that stops.

Step 2 — Read the requirement into constraints

Keyword in the problem	Constraint
stop paying for compute	reach the **deallocated** state
don't delete it	use deallocate, **not** `az vm delete`
start again tomorrow	deallocate is reversible with `az vm start`

Step 3 — Find the right verb

Guess the noun/verb and confirm in `--help`:

```
az vm --help | grep -iE "stop|deallocate|start"
```

You'll see three relevant verbs, and now you know exactly which one you want:

- `az vm stop` — stops the OS but **stays allocated** (still billing). Not this.
- `az vm deallocate` — **releases the hardware, stops compute billing.** This one.
- `az vm start` — brings a stopped/deallocated VM back.

Reference: https://learn.microsoft.com/cli/azure/vm#az-vm-deallocate

---

Step 4 — Do it

```
az vm deallocate \
  --resource-group lab-compute-vm05-rg \
  --name           dev-tool-vm
```

To bring it back tomorrow:

```
az vm start -g lab-compute-vm05-rg -n dev-tool-vm
```

Step 5 — Verify (and see the trap for yourself)

```
az vm get-instance-view -g lab-compute-vm05-rg -n dev-tool-vm \
  --query "instanceView.statuses[?starts_with(code,'PowerState/')].displayStatus" -o tsv
# -> VM deallocated     (NOT "VM stopped")

./validate.bash
```

If you had run `az vm stop` instead, that command would print **`VM stopped`** — and the validator would (correctly) fail you, because you'd still be paying. That contrast is the whole point of the exercise.

Step 6 — Troubleshooting with composure

Symptom	Likely cause	Move
Validator says state is `PowerState/stopped`	You used `az vm stop` or shut down inside the OS	Run `az vm deallocate`
Validator says `PowerState/running`	Nothing took effect, or you started it again	Re-run `az vm deallocate`
Public IP changed after start	A **dynamic** public IP is released on deallocate	Use a **Static** IP SKU if the address must persist
Need it off nightly, automatically	Manual is error-prone	Look at **auto-shutdown** (`az vm auto-shutdown`) or a scheduled automation

---

Cheat sheet

Goal	Command
Stop compute billing (keep the VM)	`az vm deallocate -g <rg> -n <vm>`
Start it again	`az vm start -g <rg> -n <vm>`
Check the exact power state	`az vm get-instance-view -g <rg> -n <vm> --query "instanceView.statuses[?starts_with(code,'PowerState/')].displayStatus" -o tsv`
Stop the OS but keep paying (the trap)	`az vm stop -g <rg> -n <vm>`
Schedule a nightly shutdown	`az vm auto-shutdown -g <rg> -n <vm> --time 1900`

Takeaway: in the cloud, "off" has a price tag unless you deallocate. Knowing the difference between *stopped* and *deallocated* is a small fact that saves real money — exactly the kind of thing that separates a cloud admin from someone who just clicks "stop."

Question 6 — Provision a Web Server with a Provisioner Script

Scenario

The team wants a web server that comes up **ready to serve** — no manual SSH, no hand-installing packages. As the cloud administrator, **provision a VM that configures itself on first boot**: a provisioner script should install and start **nginx**, and the VM should be reachable on **port 80** so that curling its public IP returns the nginx welcome page.

This is how real deployments work: the VM is described *as code* and stands itself up. `setup.bash` only checks your environment — building the VM (with its provisioner script) is the whole exercise.

Requirements

#	Requirement	Value
1	A VM exists	`dev-tool-vm` in `lab-compute-vm06-rg`, Ubuntu 26.04, `Standard_B2als_v2`
2	**nginx** is installed and running — placed there by a **provisioner script**, not by manual SSH	via cloud-init `--custom-data`
3	Inbound **TCP 80** is open	access: Allow, direction: Inbound, port: 80
4	Curling the VM's public IP returns nginx	`curl http://<public-ip>` → HTTP 200, `Server: nginx`

> A provisioner script runs on **first boot**, so give it a minute or two after the VM is created before you validate — installing nginx isn't instant.

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open solution.md.

Clean up

```
./cleanup.bash
```

Solution — Question 6: Provision a Web Server with a Provisioner Script

This question combines three skills: **provisioning** (Q1), **opening a port** (Q2), and a new one — making the VM **configure itself on first boot** with a provisioner script. That last idea, "infrastructure that stands itself up," is the doorway to real automation.

Step 1 — Read the requirement into constraints

Keyword in the problem	Constraint
configures itself on first boot	a **provisioner script** — on Azure/Linux that's **cloud-init** via `--custom-data`
install and start nginx	the script's job: `apt install nginx`, ensure it's running
reachable on port 80	open inbound TCP 80 (an NSG rule)
curl returns the nginx page	the end-to-end proof: HTTP 200 with `Server: nginx`

Two moving parts have to line up: nginx must be **installed** (provisioner script) **and** the network must **let traffic in** (port 80). Miss either and the curl fails — a great lesson in "it works on the box but not from outside."

Step 2 — What is a "provisioner script" on Azure?

When a Linux VM boots for the first time, **cloud-init** looks for user-supplied configuration and runs it. `az vm create` hands your file to cloud-init through the `--custom-data` parameter. So the "provisioner script" is just a small cloud-init file you write and pass in. Reference:

- cloud-init on Azure: https://learn.microsoft.com/azure/virtual-machines/linux/using-cloud-init
- `az vm create` (see `--custom-data`): https://learn.microsoft.com/cli/azure/vm#az-vm-create

  There's a second way — the **Custom Script Extension** (`az vm extension set --name CustomScript ...`) — which runs a script *after* the VM is already up. cloud-init is the cleaner "first boot" choice and is what we use here; knowing the extension exists is enough for now.

Step 3 — Write the provisioner script (cloud-init)

Create a file called `cloud-init.yaml`. The `#cloud-config` first line is required — it tells cloud-init how to read the file.

```yaml
#cloud-config
package_update: true
packages:
  - nginx
runcmd:
  - systemctl enable nginx
  - systemctl start nginx
```

What each part does:

- `package_update: true` — refresh the apt package index first (like `apt update`).
- `packages:` — install nginx from the distro repos.
- `runcmd:` — commands run late in boot; here we make sure nginx is enabled (starts on every boot) and running now.

On Ubuntu, installing the `nginx` package already starts it — the `runcmd` lines are belt-and-suspenders so the requirement holds even if defaults change.

Step 4 — Provision the VM with the script, then open the port

```
# 1) Create the resource group
az group create --name lab-compute-vm06-rg --location centralus

# 2) Create the VM, handing your provisioner script to cloud-init
az vm create \
  --resource-group lab-compute-vm06-rg \
  --name           dev-tool-vm \
  --image          Canonical:ubuntu-26_04-lts:server:latest \
  --size           Standard_B2als_v2 \
  --admin-username azureuser \
  --generate-ssh-keys \
  --custom-data    cloud-init.yaml        # <-- the provisioner script

# 3) Open inbound port 80 (same skill as Question 2)
az vm open-port \
  --resource-group lab-compute-vm06-rg \
  --name           dev-tool-vm \
  --port           80 \
  --priority       900
```

`--custom-data` accepts the path to your file (the CLI reads and base64-encodes it for you). cloud-init then runs it during first boot — which takes a minute or two **after** `az vm create` returns. Patience here is part of the skill.

Step 5 — Verify the way a user would: curl the public IP

```
# Get the public IP
IP=$(az vm list-ip-addresses -g lab-compute-vm06-rg -n dev-tool-vm \
      --query "[0].virtualMachine.network.publicIpAddresses[0].ipAddress" -o tsv)
echo "$IP"

# Hit it. -I shows headers; look for "Server: nginx" and "200 OK".
curl -I "http://$IP"

./validate.bash
```

If it's not up yet, wait 30–60s and try again — cloud-init may still be installing. The validator already retries for you.

Step 6 — Troubleshooting with composure

The classic failure here is "curl times out," and the discipline is to isolate **which** of the two moving parts is wrong:

Symptom	Likely cause	Move
Connection **times out**	Port 80 not open (NSG)	Confirm the rule: `az network nsg rule list -g <rg> --nsg-name dev-tool-vmNSG -o table`
Connection **refused** / empty reply	Port is open but nginx isn't running	SSH in; `systemctl status nginx`; check cloud-init ran

Symptom	Likely cause	Move
Works after a long wait	cloud-init was still installing	Normal — it runs on first boot; just wait
Never installs	Bad cloud-init (e.g. missing `#cloud-config` line, wrong indentation)	On the VM: `sudo cloud-init status --long` and `sudo cat /var/log/cloud-init-output.log`

The single most useful file when a provisioner script misbehaves:

```
# (SSH into the VM)
sudo cat /var/log/cloud-init-output.log     # shows exactly what your script did
```

That log is the difference between guessing and knowing.

Cheat sheet

Goal	Command
Pass a provisioner script	`az vm create ... --custom-data cloud-init.yaml`
Open port 80	`az vm open-port -g <rg> -n <vm> --port 80 --priority 900`
Get the public IP	`az vm list-ip-addresses -g <rg> -n <vm> --query "[0].virtualMachine.network.publicIpAddresses[0].ipAddress" -o tsv`
Test the web server	`curl -I http://<public-ip>`
Read the provisioner log (on the VM)	`sudo cat /var/log/cloud-init-output.log`
Check cloud-init finished (on the VM)	`sudo cloud-init status --long`

Takeaway: a VM that stands itself up is the first step toward infrastructure-as-code. You described the machine's desired state in a tiny file, and it built itself — and when it doesn't, `cloud-init-output.log` tells you why.

Question 1 — Deploy a Web App

Scenario

The team wants to stop managing servers for a small internal website. Instead of a VM they'll patch and babysit, they want a **platform-managed web app** — they push code, Azure runs it. As the cloud administrator, **stand up an Azure App Service web app** and prove it's live in a browser.

App Service is a **Platform-as-a-Service (PaaS)**: you don't see the VM, the OS, or the web server — Azure runs all of that. You provide the app; Azure provides the platform.

Requirements

#	Requirement	Value
1	A resource group exists	`lab-compute-app01-rg`, in your lab region
2	An **App Service plan** exists in it	Linux, a **paid** tier (e.g. `B1` Basic) — not Free (`F1`)
3	A **web app** runs on that plan	a **Node** runtime (e.g. `NODE:22-lts`)
4	The app is **reachable**	`curl https://<app-name>.azurewebsites.net` returns **HTTP 200**

> **Web app names are globally unique** — the name becomes `https://<name>.azurewebsites.net`, a public DNS host shared by all of Azure. Pick something unlikely to be taken, e.g. `dev-tool-app-<your-initials><digits>`. The validator finds *the* web app in your resource group, so the exact name is your choice.

Before you start

Work from **inside this question's folder**. Then:

```
az login
az account show
./setup.bash      # optional: preflight + shows your target
```

When you think you're done

```
./validate.bash
```

Stuck? Open solution.md.

Clean up

```
./cleanup.bash
```

Solution — App Service Q1: Deploy a Web App

The whole task is three commands. The skill is understanding the **two-layer model** of App Service — the *plan* (the rented compute) and the *app* (your code running on it) — because every other App Service task builds on that split.

Step 1 — The mental model: plan vs. app

App Service separates **where your code runs** from **the code itself**:

- **App Service plan** — the compute you rent: OS (Linux/Windows), region, and a **pricing tier** (F1 Free, B1 Basic, S1 Standard, P-series Premium) that sets CPU/RAM, instance limits, and which features you get. Think of it as the *server farm*.
- **Web app** — your application, running **on** a plan, with a chosen **runtime** (Node, Python, .NET, …). Many web apps can share one plan.

So the order is always: **create the plan → create the app on it.** That's the same "create the box, then put the thing in the box" pattern as a resource group, one level down.

Reference: https://learn.microsoft.com/cli/azure/appservice/plan and https://learn.microsoft.com/cli/azure/webapp

---

Step 2 — Read the requirement into constraints

Keyword in the problem	Constraint
platform-managed web app	App Service (PaaS), not a VM
paid tier, not Free	plan `--sku B1`, `--is-linux`
Node runtime	web app `--runtime NODE:22-lts`
reachable / curl 200	the app answers on `https://<name>.azurewebsites.net`

Why not Free (F1)? F1 is quota-limited, has no "Always On," can't scale out, and has no deployment slots — so later questions in this topic (scale-out, slot swaps) need at least Basic/Standard anyway. Basic (`B1`) costs only a couple of cents for a short lab session — just remember to run `./cleanup.bash` when you finish, since an App Service plan left running bills around the clock.

Step 3 — Which runtime string? Discover, don't guess

Runtime names have an exact format (`NODE:22-lts`). Never guess them — list them:

```
az webapp list-runtimes --os linux            # every valid --runtime value
az webapp list-runtimes --os linux | grep -i node
```

Pick a current LTS, e.g. `NODE:22-lts`.

Step 4 — Build it (three commands)

```
# 1) The resource group (the box)
az group create --name lab-compute-app01-rg --location centralus

# 2) The App Service plan (the rented compute). --is-linux picks Linux hosts.
az appservice plan create \
  --resource-group lab-compute-app01-rg \
  --name           dev-tool-plan \
  --sku            B1 \
  --is-linux
```

```
# 3) The web app on that plan. --name must be GLOBALLY unique.
az webapp create \
  --resource-group lab-compute-app01-rg \
  --plan           dev-tool-plan \
  --name           dev-tool-app-ab1234 \        # <-- change to your own unique name
  --runtime        "NODE:22-lts"
```

`az webapp create` returns JSON including `defaultHostName` — that's your public URL, `https://<name>.azurewebsites.net`. A freshly created app serves a default "your app is running" page (HTTP 200) even before you deploy code, which is exactly what we're proving here.

Step 5 — Verify the way a user would

```
# What's the public URL?
az webapp show -g lab-compute-app01-rg -n dev-tool-app-ab1234 \
  --query defaultHostName -o tsv

# Hit it
curl -I https://dev-tool-app-ab1234.azurewebsites.net     # expect: HTTP/... 200

./validate.bash
```

Step 6 — Troubleshooting with composure

Symptom	Likely cause	Move
`Website with given name ... already exists`	The web app name isn't globally unique	Pick another name (add more random characters)
`curl` hangs then 200 after a while	**Cold start** — the app spins up on first request	Normal; the validator retries
403 / "Web App is stopped"	The app or plan is stopped	`az webapp start -g <rg> -n <app>`
Plan create fails on `--is-linux` region	Rare regional capacity issue	Try another region via `LAB_LOCATION`

Cheat sheet

Goal	Command
List runtimes	`az webapp list-runtimes --os linux`
Create a plan	`az appservice plan create -g <rg> -n <plan> --sku B1 --is-linux`
Create a web app	`az webapp create -g <rg> --plan <plan> -n <unique-name> --runtime "NODE:22-lts"`
Get the URL	`az webapp show -g <rg> -n <app> --query defaultHostName -o tsv`
Test it	`curl -I https://<app>.azurewebsites.net`
Stop / start	‘az webapp stop

Takeaway: App Service trades control for convenience — no OS to patch, no web server to configure. You rent a *plan* and run an *app* on it. Hold that two-layer model and the rest of App Service (settings, scaling, slots) falls into place.

Question 2 — Configure App Settings

Scenario

The web app needs configuration: it reads its behaviour from environment variables (which database to use, which mode to run in, feature toggles). In App Service these are called **application settings** — key/value pairs Azure injects into the app's environment. As the cloud administrator, **set the app's configuration** without redeploying any code.

`setup.bash` has already provisioned a plan and a running web app for you.

Requirements

The web app in `lab-compute-app02-rg` must have these two application settings:

Setting name	Value
`ENVIRONMENT`	`production`
`FEATURE_FLAG`	`beta`

> App settings are per-app configuration, surfaced to your code as environment variables. Setting them restarts the app automatically — that's expected.

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Solution — App Service Q2: Configure App Settings

Two commands, one important idea: in App Service you change configuration **without touching code or the server**. App settings are the PaaS replacement for "SSH in and edit a config file."

Step 1 — What "app settings" are

An **application setting** is a key/value pair Azure injects into your app's process as an **environment variable**. Your code reads `ENVIRONMENT` the same way it would read any env var (`process.env.ENVIRONMENT` in Node, `os.environ` in Python). They live in the platform, not in your code — so you can change them per environment (dev/staging/prod) without redeploying.

Reference: https://learn.microsoft.com/cli/azure/webapp/config/appsettings

---

Step 2 — Read the requirement into constraints

Keyword	Constraint
set configuration, no redeploy	use app settings, not a code change
`ENVIRONMENT=production`	one key/value
`FEATURE_FLAG=beta`	a second key/value

Step 3 — Find the command

The setting lives under a web app's **config**, so the noun path is `az webapp config appsettings`. Check the verbs:

```
az webapp config appsettings --help      # -> set, list, delete
```

`set` takes `--settings KEY=VALUE` (space-separated for multiple).

Step 4 — Do it

First, what's the app called? (setup generated a unique name — find it.)

```
APP=$(az webapp list -g lab-compute-app02-rg --query "[0].name" -o tsv)
echo "$APP"
```

Set both settings in one call:

```
az webapp config appsettings set \
  --resource-group lab-compute-app02-rg \
  --name           "$APP" \
  --settings       ENVIRONMENT=production FEATURE_FLAG=beta
```

Setting app settings triggers an app **restart** so the new environment takes effect — expected, and quick.

Step 5 — Verify

```
az webapp config appsettings list \
  -g lab-compute-app02-rg -n "$APP" \
  --query "[?name=='ENVIRONMENT' || name=='FEATURE_FLAG']" -o table

./validate.bash
```

Step 6 — Troubleshooting with composure

Symptom	Likely cause	Move
Setting not showing	Typo in the key, or set on the wrong app	Re-list; confirm `$APP` is the right name
Value has spaces/special chars	Shell split it	Quote it: `--settings KEY="some value"`

Symptom	Likely cause	Move
Secret in plain text	App settings are readable by anyone with access	For real secrets, use a **Key Vault reference** instead of a literal value

Security note: app settings are fine for non-secret config. For passwords, connection strings, and API keys, store the secret in **Azure Key Vault** and put a *reference* in the app setting — the value stays in the vault, and the app reads it at runtime. That habit keeps secrets out of your CLI history and the portal.

Cheat sheet

Goal	Command
Find the app in an RG	`az webapp list -g <rg> --query "[0].name" -o tsv`
Set settings	`az webapp config appsettings set -g <rg> -n <app> --settings K1=V1 K2=V2`
List settings	`az webapp config appsettings list -g <rg> -n <app> -o table`
Delete a setting	`az webapp config appsettings delete -g <rg> -n <app> --setting-names K1`

Takeaway: configuration is a platform concern, not a code concern. App settings let you retune an app per environment in seconds — and knowing to push *secrets* into Key Vault instead is what separates a config change from a security incident.

Question 3 — Scale Out

Scenario

A marketing push is coming and the single instance behind the web app won't keep up. Rather than move to a bigger machine, the team wants **more copies of the app running behind Azure's load balancer** so traffic is spread across them. As the cloud administrator, **scale the app out to 3 instances.**

`setup.bash` has provisioned the plan and web app (currently 1 instance).

Requirements

#	Requirement	Value
1	A web app exists	in `lab-compute-app03-rg`
2	The App Service plan runs **3 instances**	worker/instance count = `3`

> "Scale **out**" (more instances) is different from "scale **up**" (a bigger tier). This question is about scaling *out*. The Basic tier supports up to 3 instances — enough for this exercise.

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Solution — App Service Q3: Scale Out

One command — but only if you know the difference between the two kinds of scaling and where the "instance count" knob actually lives (on the **plan**, not the app).

Step 1 — Scale up vs. scale out

	Scale **up** (vertical)	Scale **out** (horizontal)
What changes	a **bigger** tier (more CPU/RAM per instance)	**more instances** of the same size
CLI	change the plan's `--sku` (B1 → S1 → P1v3)	change the plan's **worker count**
Good for	a single heavy request path	lots of concurrent requests

The marketing push is about **concurrency** — many users at once — so you scale **out**: add instances and let App Service's built-in load balancer spread traffic across them. (Scaling up is a separate lever you'd reach for if each request were individually too heavy.)

Step 2 — The knob is on the PLAN

Instances are a property of the **App Service plan** (the compute), not the web app. All apps on a plan share its instances. So you update the plan's worker count. From `az appservice plan update --help` the flag is `--number-of-workers`. Reference: https://learn.microsoft.com/cli/azure/appservice/plan#az-appservice-plan-update

---

Step 3 — Do it

Find the plan, then scale it out:

```
PLAN=$(az appservice plan list -g lab-compute-app03-rg --query "[0].name" -o tsv)

az appservice plan update \
  --resource-group lab-compute-app03-rg \
  --name           "$PLAN" \
  --number-of-workers 3
```

> **Tier limits matter.** Basic (B-series) allows up to **3** instances — which is why this exercise targets 3. To go higher you'd first scale *up* to Standard (10) or Premium (30). And true **auto**scale (add/remove instances based on CPU or a schedule) requires Standard or higher — on Basic you scale manually.

Step 4 — Verify

```
az appservice plan show -g lab-compute-app03-rg -n "$PLAN" \
  --query "{tier:sku.tier, instances:sku.capacity}" -o table
# instances -> 3

./validate.bash
```

Step 5 — Troubleshooting with composure

Symptom	Likely cause	Move
Error setting workers to > 3	You're on Basic (max 3)	Scale *up* to Standard first, then scale out further
`capacity` still 1	Updated the wrong plan, or command didn't apply	Re-check `$PLAN`; re-run the update
Want it automatic	Manual scaling doesn't react to load	Use autoscale rules (needs Standard+): `az monitor autoscale ...`

Cheat sheet

Goal	Command
Find the plan	`az appservice plan list -g <rg> --query "[0].name" -o tsv`
Scale out (instances)	`az appservice plan update -g <rg> -n <plan> --number-of-workers 3`
Scale up (tier)	`az appservice plan update -g <rg> -n <plan> --sku S1`
Check tier + instances	`az appservice plan show -g <rg> -n <plan> --query "{tier:sku.tier,instances:sku.capacity}" -o table`

Takeaway: "the app is slow under load" usually means **scale out**; "each request is heavy" means **scale up**. Both are one command on the *plan* — and knowing the tier ceilings (Basic = 3) keeps you from promising capacity your tier can't deliver.

Question 4 — Zero-Downtime Release with a Staging Slot

Scenario

Production is live and you need to ship a new version **without downtime** and **without risk**. App Service's answer is **deployment slots**: a separate, fully-running copy of the app (e.g. `staging`) that you deploy and test on its own URL, then **swap** into production — Azure warms up the new version first and flips traffic over instantly.

`setup.bash` has provisioned the plan (Standard tier — slots require it) and a web app whose current release is marked `RELEASE=v1`.

Your job: roll out **v2** through a staging slot and swap it live.

Requirements

#	Requirement	Value
1	A deployment slot named **`staging`** exists on the web app	slot `staging`
2	You mark the new release in staging and **swap** it into production	prod app setting `RELEASE` = `v2` after the swap
3	Production is healthy after the swap	`curl https://<app>.azurewebsites.net` → HTTP 200

> We use the app setting `RELEASE` as a stand-in for "a new version of the code": set `RELEASE=v2` on the **staging** slot, confirm it there, then swap — after which **production** carries `v2`. (App settings move with the slot on swap unless you pin them as slot-specific.)

Get started

```
./setup.bash
```

When you think you're done

```
./validate.bash
```

Stuck? Open `solution.md`.

Clean up

```
./cleanup.bash
```

Solution — App Service Q4: Zero-Downtime Release with a Staging Slot

This is the capstone: the technique real teams use to ship without downtime. The commands are short; the value is understanding **what a slot is** and **what a swap actually does**.

Step 1 — The concept: slots and swapping

A **deployment slot** is a second, fully-running copy of your web app on the same plan, with its own hostname (`https://<app>-staging.azurewebsites.net`). You deploy the new version *there* and test it in isolation.

A **swap** doesn't copy files around. Azure **warms up** the staging instance (starts it, waits until it's responding), then **flips the routing** so the warm staging instance becomes production — and the old production becomes staging. Because the new version is already warm when traffic flips, users see **no downtime** and **no cold start**. If something's wrong, you swap again to roll back instantly.

> **Slots require Standard tier or higher.** That's why `setup.bash` used `S1`. Basic and Free have no slots. `S1` is the most expensive tier in these labs, so it's especially important to run `./cleanup.bash` as soon as you're done — a Standard plan left running for days adds up.

Reference: https://learn.microsoft.com/azure/app-service/deploy-staging-slots

---

Step 2 — About the "new version" in this exercise

In reality you'd deploy new *code* to the staging slot. To keep the focus on the slot mechanics (and make the result checkable from the CLI), we represent "the new version" with an app setting: **`RELEASE=v2`**. Production starts at `RELEASE=v1`; after a correct swap, production reads `RELEASE=v2`. App settings travel with the slot on swap unless you pin them as slot-specific — which is exactly what lets a swap carry your release forward.

Step 3 — Do it

Find the app, then run the release:

```
APP=$(az webapp list -g lab-compute-app04-rg --query "[0].name" -o tsv)

# 1) Create the staging slot (a clone of production's config)
az webapp deployment slot create \
  --resource-group lab-compute-app04-rg \
  --name           "$APP" \
  --slot           staging

# 2) Put the new version on STAGING only (note --slot staging)
az webapp config appsettings set \
  --resource-group lab-compute-app04-rg \
  --name           "$APP" \
  --slot           staging \
  --settings       RELEASE=v2

# 3) (You'd test https://$APP-staging.azurewebsites.net here.)

# 4) Swap staging into production - warm-up + instant flip
az webapp deployment slot swap \
  --resource-group lab-compute-app04-rg \
  --name           "$APP" \
  --slot           staging \
  --target-slot    production
```

After the swap, **production** carries `RELEASE=v2` and the old `v1` now lives in the `staging` slot (ready to swap back if you need to roll back).

Step 4 — Verify

```
# Production now reports v2
az webapp config appsettings list -g lab-compute-app04-rg -n "$APP" \
  --query "[?name=='RELEASE'].value | [0]" -o tsv        # -> v2

# Production is healthy
curl -I "https://$(az webapp show -g lab-compute-app04-rg -n "$APP" --query defaultHostName -o
 ↪  tsv)"

./validate.bash
```

Step 5 — Troubleshooting with composure

Symptom	Likely cause	Move
`slot create` fails	Plan isn't Standard+	Slots need S1+; scale the plan up first
After swap, prod still `v1`	You set `RELEASE=v2` on production, not on `--slot staging`, then swapped the unchanged value	Set it on staging, then swap
Setting didn't move on swap	It was marked as a **slot setting** (sticky)	Sticky settings stay put by design; unpin it if it should travel
Need to roll back	Bad release reached prod	Swap again — the previous version is sitting in `staging`

Sticky settings are a feature, not a bug: things that must differ per slot (e.g. a staging database connection string) are pinned so they *don't* follow the swap. Everything else travels — which is how your release moves forward.

Cheat sheet

Goal	Command
Create a slot	`az webapp deployment slot create -g <rg> -n <app> --slot staging`
Configure a slot	`az webapp config appsettings set -g <rg> -n <app> --slot staging --settings K=V`
Swap into production	`az webapp deployment slot swap -g <rg> -n <app> --slot staging --target-slot production`
List slots	`az webapp deployment slot list -g <rg> -n <app> -o table`
Roll back	swap again (previous version is now in `staging`)

Takeaway: slots turn a scary "deploy to production" into a reversible, zero-downtime **swap**. Warm up the new version out of the traffic path, flip when it's ready, and keep the old one one command away in case you need it back.

Part 3 — Practice Exercises

Practice Exercises — Virtual Machines

These are practice problems: **questions only** — no `setup.bash`, no `validate.bash`, no `solution.md`. The graded questions (1–6) walked you through the skills; these are the reps that turn understanding into muscle memory.

Work them the same way a cloud admin works any ticket:

1. **Read the scenario into a checklist** of concrete constraints.
2. **Find the reference** — guess `az <noun> <verb>`, confirm in the docs.
3. **Tame the flags with `--help`** — required first, then match the rest to your checklist.
4. **Build the command incrementally**, then **verify with your own eyes.**

Some questions deliberately don't hand you the exact image name or size — finding those with `az vm image list -o table` and `az vm list-sizes -l <region> -o table` is part of the exercise.

How to check your own work (no validator here)

You are the validator now. The commands that tell you the truth:

```
az vm show -g <rg> -n <vm> -o jsonc                        # the full definition
az vm show -g <rg> -n <vm> --query <jmespath> -o tsv       # one specific field
az vm get-instance-view -g <rg> -n <vm> \
  --query "instanceView.statuses[?starts_with(code,'PowerState/')].displayStatus" -o tsv
az vm list-ip-addresses -g <rg> -n <vm> -o table           # public IP
curl -I http://<public-ip>                                 # web reachability
```

Ground rules

- Use your **own resource groups** — a good convention is `practice-vmNN-rg` so they're easy to find and delete. Put everything for one exercise in one group.
- These create **real, billable resources.** Clean up the moment you finish each:

  ```
  az group delete --name <your-rg> --yes --no-wait
  ```

- Region is your choice; if a size or image isn't offered where you are, pick another region or another size — that decision *is* the skill.

Exercise 1 — Provision with tags

Provision a Linux VM for a short-lived experiment, labelled so finance can tell what it's for.

Done when:

- A VM named `web-01` runs **Ubuntu 24.04 LTS** (find its image alias yourself).
- Size is `Standard_B2als_v2` (a small, widely-available burstable size).
- The VM carries two **tags**: `env=practice` and `owner=<your-name>`.
- Admin user `azureuser`, SSH-key auth.

Exercise 2 — Provision, then connect

Prove you can actually get onto a machine you built — half of administration is logging in to look around.

Done when:

- You provision an Ubuntu 26.04 VM named `jump-01`.
- You retrieve its **public IP** from the CLI (not the portal).
- You **SSH in** as `azureuser` and run `hostnamectl`, confirming the OS is Ubuntu 26.04.

Exercise 3 — Open HTTPS, but only to you

A service listens on **443**, but it must be reachable **only from your own workstation's IP**, not the whole internet.

Done when:

- An existing VM has an inbound rule allowing **TCP 443**.
- The rule's **source** is restricted to your current public IP (hint: find yours, then look for a `--source-address-prefixes`-style option).
- No other new ports are opened.

Exercise 4 — Resize for more memory

A data-processing tool is running out of RAM, not CPU. Give it more memory — while keeping to a small, widely-available size.

Done when:

- An existing `Standard_B2als_v2` VM (2 vCPU / 4 GB) is resized to `Standard_B2as_v2` (2 vCPU / **8 GB**) — same burstable family, double the RAM. Confirm it's a valid target first with `az vm list-vm-resize-options`.
- The VM is **running** afterward.
- You can explain (to yourself) when you'd reach for more memory vs. more vCPUs — and why sticking to a widely-available family avoids capacity headaches.

Exercise 5 — Attach two data disks

A database needs its **data** and its **logs** on separate disks.

Done when:

- An existing VM has **two** managed data disks attached: one **64 GiB** (`db-data`) and one **32 GiB** (`db-logs`).
- Each disk sits on a distinct **LUN**.
- You can list both from `az vm show ... --query "storageProfile.dataDisks"`.

Exercise 6 — Schedule a nightly shutdown

The team keeps forgetting to turn off a dev box. Automate it.

Done when:

- An existing VM has an **auto-shutdown** schedule set for **19:00** in your time zone (find the right `az vm` sub-command).
- Separately, practice the manual money-saver: **deallocate** the VM, confirm the power state is `deallocated` (not merely `stopped`), then **start** it again.

Exercise 7 — A different web server, with a custom page

Repeat the provisioner-script skill, but with **Apache** instead of nginx, and serve your own content.

Done when:

- You provision a VM whose **cloud-init** provisioner installs **Apache** (`apache2`) and writes an `index.html` containing the text `Hello, Cloud Admin`.
- Port **80** is open.
- `curl http://<public-ip>` returns your custom `Hello, Cloud Admin` text (hint: cloud-init's `write_files:` section can create the page).

Exercise 8 — A Windows VM with RDP

Not everything is Linux. Stand up a Windows Server and make it reachable.

Done when:

- You provision a **Windows Server 2022** VM (find the image alias with `az vm image list -o table`).
- You set an admin username and a **strong password of your own choosing** (you enter it — never share it).
- Inbound **RDP (TCP 3389)** is open so the VM could be reached with a remote desktop client.

Exercise 9 — Snapshot a disk before a risky change

Before you make a change you can't easily undo, take a backup.

Done when:

- You take a **snapshot** of an existing VM's **OS disk** (hint: get the OS disk's ID from `az vm show`, then `az snapshot create`).
- You can list the snapshot with `az snapshot list -g <rg> -o table`.
- You can explain how you'd create a new disk *from* that snapshot to recover.

Exercise 10 — Capstone: deploy a container on boot

Combine everything: a VM that installs Docker and serves a container — all from its provisioner script.

Done when:

- You provision a VM whose **cloud-init** provisioner installs **Docker** and runs a container that serves HTTP on port **80** (for example the `nginxdemos/hello` image, or `docker run -d -p 80:80 nginx`).
- Port **80** is open.

- `curl http://<public-ip>` returns the container's page.
- Bonus: SSH in and run `docker ps` to see your container running.

When you can do all ten without notes

You've built real competency in provisioning and operating Azure compute. That's the goal — not memorizing commands, but knowing how to find, build, verify, and troubleshoot any of them. Next stop in this lab: other compute resources beyond the virtual machine.

Practice Exercises — App Service

Practice problems for App Service: **questions only** — no `setup.bash`, `validate.bash`, or `solution.md`. The graded questions (1–4) walked you through the skills; these are the reps.

Work them like any ticket:

1. **Read the scenario into a checklist** of concrete constraints.
2. **Find the reference** — guess `az <noun> <verb>` (`az webapp ...`, `az appservice plan ...`), confirm in the docs.
3. **Tame the flags with `--help`** — required first, then match to your checklist.
4. **Build the command incrementally**, then **verify with your own eyes.**

Some questions leave specifics for you to discover — the exact runtime string (`az webapp list-runtimes --os linux`), an available tier, etc. That discovery *is* the exercise.

How to check your own work (no validator here)

You are the validator now:

```
APP=$(az webapp list -g <rg> --query "[0].name" -o tsv)     # find your app
az webapp show -g <rg> -n "$APP" --query defaultHostName -o tsv
curl -I https://<app>.azurewebsites.net                     # is it serving?
az webapp config show           -g <rg> -n "$APP" -o jsonc  # runtime, alwaysOn, httpsOnly...
az webapp config appsettings list -g <rg> -n "$APP" -o table
az appservice plan show -g <rg> -n <plan> --query "{tier:sku.tier,instances:sku.capacity}" -o table
```

Ground rules

- Use your **own resource groups** — e.g. `practice-appNN-rg`. One group per exercise.
- Web app names are **globally unique** — add random characters so yours is free.
- These create **real, billable resources.** Clean up the moment you finish:

  ```
  az group delete --name <your-rg> --yes --no-wait
  ```

Exercise 1 — Deploy with a different runtime

Not every app is Node. Stand one up on a different stack.

Done when:

- You create a plan (Linux, a paid tier) and a web app running a **Python** (or **.NET**) runtime — find the exact `--runtime` value with `az webapp list-runtimes --os linux`.
- `curl https://<app>.azurewebsites.net` returns HTTP 200.

Exercise 2 — Deploy your own code

A default page isn't an app. Ship actual content and see it served.

Done when:

- You deploy a tiny site/app of your own to a web app (hint: `az webapp up` from a folder, or zip deploy with `az webapp deploy --src-path`).
- `curl https://<app>.azurewebsites.net` returns **your** content, not the default landing page.

Exercise 3 — App settings and a connection string

Real apps need config *and* a database connection — stored in the right places.

Done when:

- The web app has app settings `APP_MODE=demo` and `LOG_LEVEL=info`.
- The web app has a **connection string** named `Default` (type e.g. `SQLAzure`) set via `az webapp config connection-string set`.
- You can list both back and explain why a connection string is stored separately from a plain app setting.

Exercise 4 — Scale up and turn on Always On

Some workloads shouldn't be allowed to go idle.

Done when:

- You **scale up** an existing web app's plan from Basic to **Standard** (`S1`).
- You enable **Always On** (`az webapp config set --always-on true`) so the app isn't unloaded when idle.
- You can explain the difference between this (scale **up**) and Question 3 (scale **out**).

Exercise 5 — Lock it down and watch it

Security posture and observability — two things every admin owns.

Done when:

- You enable **HTTPS-only** on the web app so plain HTTP is redirected (`az webapp update --set httpsOnly=true`).
- You confirm an `http://` request is redirected/refused while `https://` works.
- You turn on application logging and **stream the logs live** (`az webapp log config ...` then `az webapp log tail`), and trigger a request to see it appear.

When you can do all five without notes

You can stand up, configure, scale, release, secure, and observe an App Service app — the full lifecycle of a platform-managed web workload. That's the second compute type in your toolkit, alongside virtual machines.